3. Cloud project

Table of Contents

- Overview
- Unit Learning Outcomes assessed
- Attachments
- Understanding the assignment
- AWS
- What you need to do
  - Patterns
  - Learning on your own
  - Front-end applications
- Submission
  - The report
  - The demonstration video
  - Gradescope
- GenAI
- Extensions, Access Equity Plans and other special circumstances
- CRA
- I don't know how to start
  - Py tutor
  - Wallpaper of the day
  - Web transcoder
- FAQ
- Feedback and moderation
- Due date
- Change log

Overview

> You will build a cloud application that makes use of multiple cloud compute services and an appropriate cloud architecture, designed to be cost effective and scalable. You will write a professional report describing your work.

This assignment is designed to be broad. There is no fixed problem statement and no scaffolding. The scope, the problem you solve, and the services you choose are all up to you. You may solve any problem, as long as it gives you a genuine reason to use several cloud services together and to demonstrate scaling. That freedom is deliberate: this assessment is about your *judgement* in selecting and integrating cloud services, not about reproducing a worked example. You are also responsible for keeping that scope bounded and achievable.

You should not continue working on Strobe. It has been left behind with the conclusion of assignment 2.

This assessment item is eligible for the 48 hour extension. You can read more information about this on the HiQ website (https://qutvirtual4.qut.edu.au/group/student/study/assignments/submitting/late-assignments-and-extensions) .

Unit Learning Outcomes assessed

- ULO1 Discuss the elastic nature of cloud technologies and business models and their application in technical, commercial and sustainability contexts.
- ULO2 Analyse application design and implementation requirements to select suitable cloud infrastructure and software services from a range of XaaS offerings.
- ULO3 Design and implement scalable cloud applications using industry standard languages and APIs, deployed on a public cloud infrastructure and leveraging a range of cloud services.
- ULO4 Investigate the uses, characteristics, and implementation requirements of cloud services on your own from vendor documentation, code samples, and other online sources.
- ULO5 Communicate in written, verbal and visual forms about cloud applications, services, and architectures.

Weighting	Group or Individual	How I will be assessed
40% of final grade	Individual	Grade out of 40

Attachments

- Template of infrastructure YAML: **submission-a3-infra.yml (https://canvas.qut.edu.au/courses/23778/files/9335720/download)**

Understanding the assignment

As we have emphasised throughout CAB432, this unit is not primarily about software development. While this assignment requires you to write code and deploy infrastructure, the marks are awarded for demonstrating your understanding of how and why cloud services work together. Your application provides the context in which you demonstrate that understanding, rather than being the primary focus of the assessment.

You should choose a project that interests you and, ideally, one that demonstrates skills or experience that would be valuable on your resume. The choice of application is ultimately yours,

provided it gives you sufficient scope to demonstrate the cloud concepts and architectural decisions covered in the unit.

The key question throughout the assignment should therefore be: why does this need to run in the cloud, and why have you designed it this way?

At a high level, you need to pick a problem of suitable complexity: something that genuinely benefits from several cloud services working together and can be stressed to demonstrate scaling.

You then design a scalable, cost effective architecture and justify every key decision against requirements such as scalability, security, reliability, maintainability and cost.

Once designed, you implement and deploy it on AWS, and demonstrate that it scales.

Finally, you will write a professional report that explains and justifies everything, and includes the supporting artefacts (architecture diagram, costing, sustainability discussion).

This is an individual assignment. Choose something that you can design, build and report by yourself.

AWS

You remain scoped to the CAB432 shared account, in `ap-southeast-2`. The `purpose` tag must be set to `assessment 3`. Use 'AWS Resource Explorer' in the web console to find all of your existing cloud resources, and retag any older `purpose` resources you reuse so they are not deleted.

If you have not yet presented Assignment 2, keep your resources tagged as `purpose=assessment 2` and relabel them after you have finished the presentation. The due date for assignment 3 is weeks after assignment 2 presentations have finished, so you have plenty of time to do this.

There is more detail about the reuse of cloud resources in the FAQ at the bottom of this page.

You should also keep it low-cost: you are being assessed on cost-effective design, so do not provision large, always-on resources, and scale-to-zero and pay-per-use patterns are strongly encouraged. Resource names should include your studentID.

You are free to select from the full range of AWS services that we have already given you access to. We expect your solution to draw on services we have covered plus features you investigate yourself.

What you need to do

Even though you have been learning cloud computing via AWS, the unit is about teaching you design patterns and cloud concepts. By the end of CAB432, you should leave the unit knowing how to *design* cloud systems then *implement* that design. The details you emphasise should focus on the underlying concepts that are vendor-agnostic, rather than the specifics of the AWS services.

Patterns

In this assignment, you should consider, use and discuss the cloud computing design patterns and concepts covered in CAB432.

Auto-scaling and caching are particularly important for this assignment because they are topics that you have not previously demonstrated. Lambda's built-in scaling does not count as an implementation of auto-scaling for this assignment.

Choose at least eight from the following categories:

At least one type of data storage and persistence:

- Structured data store
- Unstructured data store

At least one type of caching:

- Edge caching
- In-memory caching

At least one type of service-to-service communication:

- API gateway
- Message queues
- Publish-subscribe messaging
- Polling
- Request callback

At least one type of compute:

- Serverless computing
- Virtual machines
- Containers

At least one type of scaling:

Note: Lambda is not included here

- Request-based scaling (e.g. new containers or VMs are started in response to user requests, e.g. start up a new Minecraft server, when a user requests)

- Metric-based scaling (e.g. containers or VMs are started/stopped in response to changes in a metric, such as average CPU usage or number of queued tasks per worker)

At least one type of load distribution:

Note: Lambda is not included here

- Dispatcher-push: a load balancer or similar forwards tasks/requests to workers
- Worker-pull: workers retrieve tasks from a queue or similar

Any of:

- Microservices
- Event-driven architectures
- Orchestration
- Identity, security and role-based access control
- Configuration management
- Interfaces such as CLI, WebUI, API and WebSocket
- Networking and DNS
- Monitoring, including logs, metrics and costing
- Managed versus unmanaged services
- AI/ML services and systems
- Garbage collection
- Public versus private access at the application layer
- Stateless and/or stateful design
- Fault tolerance and resilience
- Garbage collection
- Proxies and reverse proxies
- Resilience patterns, such as retries, timeouts and circuit breakers

Learning on your own

A distinctive feature of this assignment is that some of the services, features of services, and patterns you may use are introduced in lectures but deliberately *not* covered in practicals. Investigating them yourself is part of what is being assessed (ULO4), and you should make this learning visible in your report. You might apply the **AWS Well Architected Framework (https://aws.amazon.com/well-architected/)** to your design, or adopt managed observability or discovery tools. These are just examples; you do not need to use any of these but instead find something that interests you. Choose something relevant to your solution and demonstrate that you can learn and apply them from vendor documentation and other sources.

Front-end applications

A front-end could be a web app (e.g., React), a desktop app (e.g., Windows desktop app), or something else that is typically considered a client-facing interface. Within reason, a front-end app could also be something that a developer would experience. For example, an IDE extension (e.g., for VSCode), an API collection in an API testing tool (e.g., Postman), a TUI (e.g., built using Textual in Python), etc.

We are not looking for extensive functionality in your front-end application. The purpose of the front end is to provide an appropriate interface to your cloud-based system and demonstrate how the different components of your architecture work together.

Your front end should therefore provide sufficient functionality to interact with and demonstrate the key features of your application, but you do not need to spend a large amount of time building a polished or feature-rich user interface. In particular, the complexity of the front end is not what we are assessing. We are more interested in the variety, integration and appropriate use of cloud services in your overall architecture.

You could use GenAI to assist with developing your front end. This is a good opportunity to use AI to generate boilerplate code, suggest UI components, or help you implement straightforward functionality. However, you should understand the code you submit and be able to explain how the front end interacts with the rest of your cloud architecture.

Submission

Gradescope will again be used, in the exact same way it was used for Assignment 2.

In this assignment, the primary deliverable is a written report, accompanied by a working system and a short demonstration video.

You will submit three things.

1. The written report as PDF. Submit this to Canvas.
2. A demonstration video (4 to 5 minutes) showing that your system works, including your evidence of scaling, caching data, and asynchronous behaviour. This is "look, it works" evidence, not an explanation. The explanation belongs in the report. Submit this to Canvas.
3. A submission YAML file declaring your cloud resources, and your application source code. Submit these to Gradescope.

The report

The report is the main written deliverable and is where you communicate your design reasoning, implementation decisions and technical outcomes. It should be a professional, well-structured engineering report suitable for a technical stakeholder or client. The report should be no longer than 30 pages, including diagrams, and excluding references and appendices.

The report should describe your actual system, rather than providing a generic description of cloud computing concepts. We are looking for a highly professional document that could be handed to a cloud engineering team or technical stakeholder. Use specific architectural descriptors, project-specific evidence and honest reasoning. Clearly explain what you built, why you made particular decisions, the trade-offs you considered, and the limitations of your approach. Avoid generic prose that could describe any cloud application.

There are three core sections:

1. **Vendor-agnostic architecture:** What did you design, and why?
2. **Vendor-specific implementation:** How did you realise that design in AWS?
3. **Application implementation:** How does the application actually use those components to perform work?

Diagrams should be created digitally and be clear, accurate and easily readable within the report. They should directly represent your system and support the accompanying discussion. When a diagram is placed on an A4 page at its intended size, the text within the diagram should remain legible at approximately 12 pt. If a diagram is too complex to remain readable at this size, split it into multiple focused diagrams rather than shrinking it.

Consider using tools such as:

- **CloudCraft** (https://www.cloudcraft.co/) has a diagram editor that can be used for free
- **draw.io** (https://www.drawio.com/) has an AWS shapes set (use one of the AWS templates or click *+More Shapes* and add the AWS shapes sets.) See **draw.io blog post** (https://www.drawio.com/blog/aws-diagrams) for more info.
- **Drawing and diagramming tools** (https://aws.amazon.com/architecture/icons/) lists several other options.

The structure of the document is as follows:

Title page

- Title: CAB432 Assignment 3 2026
- Subtitle: the name of your application
- Your name and student ID
- Declaration of AI use and other acknowledgements

Executive summary

On a single page, summarise your application, its purpose and the key aspects of your cloud architecture. Briefly describe what the report covers and highlight the major architectural decisions and cloud computing patterns demonstrated.

Table of contents

Self-explanatory.

Introduction

Your report should open with an introduction and problem statement that explains what you built and why, and the functional and non-functional requirements you identified. You may wish to present this information as user stories or in the form of other project scoping/management methods you know.

Briefly, show the core features of your app using a few screenshots. The font size in each screenshot must appear at 12pt after being pasted into the document. You may make the page landscape if that makes sense. Each screenshot must be accompanied by a caption and referenced somewhere in the report.

Vendor-agnostic architecture

Begin with a high-level architecture diagram that describes your designed system without reference to specific cloud providers or vendor products. The idea is that your design could be implemented using the technologies of any cloud vendor.

The diagram should illustrate the major components and architectural concerns, including:

- Computing
- Storage
- Communication and networking
- Identity and access management
- Other relevant infrastructure components

You should describe your design and selected patterns. Include a reflection on scaling, your asynchronous workflows, and your observability setup, along with any limitations and how you would address them.

Discuss your reasoning for the major choices in the cloud architecture. This is largely based on your selection from the "Patterns" section of this dection.

Include a discussion of your key decisions: why these types of services, how they integrate, and the trade-offs you considered, together with your approach to scalability, security, reliability and maintainability.

Vendor-specific implementation

Having established your vendor-agnostic design, explain how you implemented that architecture using AWS.

Present a second, more detailed architecture diagram that is specific to your AWS implementation. Use appropriate AWS service icons and clearly identify the services used for computing, storage, networking, communication, identity and other relevant functions.

This diagram should describe the implemented system at a lower level of abstraction, showing how the selected AWS services are connected and how they support the overall architecture.

Include a discussion of your key decisions: why these AWS services, how they integrate, and the trade-offs you considered, together with your approach to scalability, security, reliability and maintainability.

Application implementation

Having described the proposed architecture and its AWS implementation, explain how the application itself makes use of the underlying infrastructure. Focus on the application logic, data flows and interactions between services, rather than repeating the infrastructure-level discussion.

For example,

- What data does your user and application generate, process and store? Which cloud services are used to store it, and why were those services selected?
- What data is exchanged between components and microservices? Describe the communication mechanisms, message formats and data schemas. Include examples of data objects where appropriate.
- How does data flow through the application from the initial user request to the final result? Explain how the different components work together to fulfil a task.
- How does your application detect, handle and recover from errors that occur in the underlying infrastructure? Consider failures in individual services, invalid input, network issues and unsuccessful requests.
- How does the application behave when demand increases or a component becomes unavailable?

Do not paste code into this section. Instead, explain the key interactions, workflows and design decisions using figures such as architecture diagrams, sequence diagrams and algorithms where appropriate.

Cost and sustainability

You should include a section on cost and sustainability, using the AWS Pricing Calculator and citing your figures to cover how your design stays cost effective, how scaling affects cost, and the sustainability implications of your architectural choices.

Although cost optimisation is not the primary focus of this project, this criterion involves estimating the monthly cost of your solution, assuming an average demand of 50 concurrent users.

You will need to estimate what 50 concurrent users looks like from the perspective of resources utilisation. For this you can do some experimentation by using the application yourself for a while (say, 15 minutes) and see what impact this has on metrics like average CPU utilisation, RAM usage, bucket storage, and so on. AWS has a lot of metrics that you can access through CloudWatch or the monitoring tab on the details page for many services. It's not important that this estimate be super accurate, but you should try to get the order of magnitude.

If your application won't currently support 50 concurrent users, choose smaller number and specify this in your report.

You should use the AWS Pricing Calculator.

- **https://calculator.aws/ (https://calculator.aws/#/)**
- You will need to provide the public share link that is generated by clicking "Share" in the summary of your estimate.

In your report, provide a summary which includes the total price for each AWS service, along with the public link.

If you wish to provide a cost breakdown from a different tool, please ask the teaching team.

Reflection

You should reflect on learning on your own, describing the cloud services, patterns, or tools you had to investigate yourself (those not covered in practicals).

Your reflection could address:

- Which unfamiliar services, technologies or architectural patterns did you research, and why were they relevant to your project?
- How did you use the reference documentation to evaluate or improve your design?
- How did your investigation change or shape your architectural decisions? Identify specific examples where research led you to adopt, modify or reject an approach.
- What difficulties did you encounter when learning or implementing unfamiliar technologies, and what did you learn from addressing them?
- Based on your experience, what would you investigate further or approach differently in a future iteration?

The focus should be on what you learned, how you learned it, and how that learning influenced your technical decisions, rather than simply listing the services or resources you consulted.

References and appendices

Include references to documentation and other sources used throughout your report.

Use appendices for supporting material that is useful for understanding or evaluating your work but would interrupt the flow of the main report. For example, this might include additional diagrams, configuration information, detailed test results or supplementary evidence.

Do not use appendices to avoid discussing important information in the main report. Key design decisions, evidence and explanations should appear in the relevant sections of the report.

The demonstration video

Important! Ensure that your video is actually submitted. You need to click the *Submit* button after uploading your video.

- Your video must be submitted to Canvas
- Your video must be 4 to 5 minutes
- Your video must follow the structure below and demonstrate your application's functionality as requested
- Your video must be a screen capture at a high enough quality that the marker can make out details such as text. A recording from your phone is not acceptable.

- When possible, increase the font size of what is on your screen by zooming in (CTLR and +)
- The purpose of the video is to *demonstrate* functionality, not explain it. Keep things brief in the video; you only need to describe what you are demonstrating.

Creating your video:

- You can record short videos separately and edit them together
- If some operation takes a long time (eg. uploading a video) you can edit it down to save time
- **OBS Studio (https://obsproject.com/)** is a reasonable choice for screen capture, but you can use other software if you like.
- Canvas has a built in option for screen capture which you can access through the submission page. Choose the *Record media* option.

Gradescope

This is largely the same as assignment 2. In assignment 3, you will submit a YAML file listing your cloud resources, **and all application source code**.

You can find the YAML template at the top of this page. Do not forget to submit your application source code.

When checking your infrastructure YAML file, Gradescope will only verify that the resources you declare exist, are correctly tagged, and are in-region. It will not test your system end-to-end. Marks for architecture, implementation, and communication come from your report and your presentation.

The `cloudInfrastructure` list should contain every cloud resource your solution depends on. The `type` field describes the kind of resource (for example, *Lambda function*, *EC2 Auto Scaling group*, *Application Load Balancer*, or *CloudWatch dashboard*), while the `arn` field should contain the resource's unique identifier. This format is intentionally flexible and is not limited to AWS services. Each resource must also include a one-sentence plain-language description explaining its purpose in your solution.

Below is a sample. Add or remove as many resources as needed.

```yaml
cloudInfrastructure:
  - type: EC2 Auto Scaling group
    arn: <arn-or-other-identifier>
    description: <one-line description>

  - type: Application Load Balancer
    arn: <arn-or-endpoint>
    description: <one-line description>

  - type: Lambda function
    arn: <arn>
    description: <one-line description>

metaData:
  qutUsernameTag: <qut-username@qut.edu.au>
```

GenAI

The ethical and responsible use of generative artificial intelligence (GenAI) tools is authorised in this assessment. The report must be drafted entirely by you and AI can only be used to refine your writing style.

If we need to seek clarification about your work, we may ask you to attend an interview with the unit coordinator where you will need to explain your design and implementation.

Extensions, Access Equity Plans and other special circumstances

The due date for the assignment applies to all three items that need to be submitted. Please submit by the due date, or seek a formal extension through the usual process.

If you have an Access Equity Plan (AEP) that has adjustments to oral presentations, please contact us at **cab432@qut.edu.au (mailto:cab432@qut.edu.au)** . If you are only using your AEP for an extension, then please follow the process for formal extensions.

If you have special circumstances not covered by any of the above cases, or if you have other concerns, please contact the teaching team at **cab432@qut.edu.au (mailto:cab432@qut.edu.au)** and include "A3" in the subject line.

CRA

The rubric is available via the Assessments tab on Canvas. Be sure you have carefully considered how to maximise the marks you can achieve.

The assessment is designed to evaluate both what you build and how you justify it. Marks are awarded across four areas: analysing the problem and identifying suitable cloud solutions; designing and justifying an appropriate cloud architecture; implementing and evaluating the system, including its scalability and operation in the cloud; and communicating your work through a professional report that uses evidence, reflection, and technical reasoning.

A working application is important, but the assessment is not simply a demonstration of functionality. You should be able to explain why your solution is designed the way it is and how your design addresses the requirements of the assignment. Strong responses will show that you

understand not only what your application does, but why particular architectural approaches, workflows, and cloud services were appropriate for the problem.

Remember that the assessment criteria are applied in the context of the assignment brief. Your solution should therefore clearly demonstrate how you responded to the requirements of the task and why your chosen approach was appropriate for those requirements.

I don't know how to start

We encourage you to choose your own project to design and implement. Take the time to brainstorm your ideas rather than rushing into a decision. The project you choose will provide the foundation for demonstrating your cloud engineering skills throughout the assignment.

Though, if you are unsure how to proceed, we have provided some ideas below. You could start here and build on them.

We have provided these examples to build on top of different architectual designs.

Py tutor

Py tutor is a web app where users can edit and run Python applications, similar to **Online Python (https://www.online-python.com/online_python_compiler)** or **W3Schools Python Tryit Editor (https://www.w3schools.com/python/trypython.asp?filename=demo_compiler)** . It has a code editor, a console where the user can interact with their running applications (see text output, type input), and a chat pane where the user can chat with an LLM-based tutor. The tutor has tools for examining the code and the console, but it does not have tools for editing the code. The user can ask the tutor questions, for example what the problem with the code is, etc..

Like the W3 schools and Online Python offerings, Py tutor can be used without signing in, so no authentication is required. However, Py tutor does allow users to open a previous session. There is a button which generates a link with a unique identifier as a parameter. When they go to that link the editor is populated with their previous code.

Securtity is an important aspect of running Python code from unknown users. A naive approach would make it trivial for an attacker to act with the same permissions as the server. Fortunately, it's relatively easy to run code in a sandbox, for example by creating a new container to run the code in. Of course this does require some additional work to keep track of containers, connect user requests up to the correct container, and stop the container when the session is inactive for some time.

Wallpaper of the day

Many people like to change their computer's wallpaper frequently, but it can be annoying to find images that are the correct size, especially for screens that aren't the usual 1920x1080 resolution. Wallpaper of the day (WotD) is a web app which solves this problem. Give it a URL to an image (from one of WotD's list of approved sites, such as NASA's photo of the day gallery), plus your desired size, and it will resize the image for you!

WotD uses advanced caching mechanisms to ensure that you never have to wait, even when all of our dedicated users are downloading their new wallpapers each morning. First, we cache source images locally over the long term so that images are there, ready to be resized, and we'll still have them if the original source takes the image down. We also use an ultra-fast in-memory cache so that we can quickly provide custom sizes of recent viral images. Finally, we cache the final resized images at the edge -- close to you! -- so that our most popular images and sizes download in milliseconds.

Not sure what wallpaper you want? We'll select one from our long-term cache randomly for you and resize it to your specifications.

Web transcoder

There are a lot of video formats, and people take lots of videos. But sometimes your phone doesn't record in a format that is suitable for sharing with others directly. Web Transcoder is a service where users can simply drop their video onto the web page and after a short wait they can download their video in H.264 format, perfect for sharing.

Web Transcoder uses the stardard `ffmpeg` tool for transcoding. It is responsive to load, allocating more transcoding workers when there are a lot of videos waiting to be transcoded, so that users never have to wait too long, no matter how busy the site becomes.

Web Transcoder keeps your transcoded videos around for 2 hours, after which it is deleted for your privacy.

FAQ

I can only have one API Gateway, how can I work on A3 without breaking A2?

You can add more endpoints to your API by having a prefix on new paths. For example, add `/a3`:

- `/a3/foo`
- `/a3/bar`
- `/a3/baz`

I only have one Cognito user pool, what do I do?

Just keep using that same pool. No need to relabel.

I am running out of DynamoDB tables, what do I do?

Add a sort key, if you don't have one already.

The table structure is extremely flexible. We are okay with you storing objects that don't share the same fields.

Can I reuse my ECS cluster?

Yes. No need to relabel.

Can I delete my old resources?

Yes, please do! Unless you still need them for the A2 presentation.

Feedback and moderation

Given this is the last assessment item for the unit, as per QUT MOPP, we are unable to release your mark until your final grade has become available to you. Read about that on HiQ **here (https://qutvirtual4.qut.edu.au/group/student/study/grades-reviews-and-academic-issues/receiving-your-final-grades)**). After final grades have been released, you may request a review. Read about that **here (https://qutvirtual4.qut.edu.au/group/student/study/grades-reviews-and-academic-issues/reviews-and-appeals/review-of-final-unit-grade)** .

The reason for each choice of mark is usually self-evident. The marker will include some written feedback about your performance.

All staff who are assessing your work meet to discuss and compare their judgements before marks or grades are finalised.

Due date

This assignment is due during the **central examination period**.

This assignment is eligible for the 48-hour late submission period and assignment extensions.

Change log

- 28 Septembmer 2026

> Initial deployment for CAB432_26se2.